

An Efficient Deepfake Face Detection Model Based on CNN and MobileNetV2 Transfer Learning

*A Project Report submitted in partial fulfilment of the requirements
for the award of the degree of*

Master of Computer Applications

by

Jatin	12584200089
Aditya Gaur	12584200012
Piyush Sharma	12584200138
Vipin Kumar	12584200209

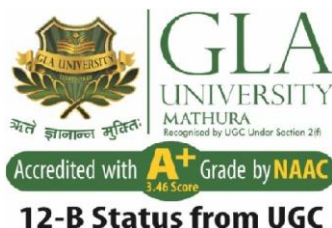
Group No. 33

Under the Guidance of

Dr. Sachendra Singh Chauhan

Department of Computer Engineering & Applications

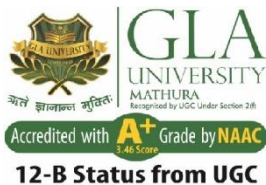
Institute of Engineering & Technology



GLA University

Mathura- 281406, INDIA

April, 2026



Department of Computer Engineering and Applications
GLA University, 17 km Stone, NH#2, Mathura-Delhi Road,
P.O. Chaumuhan, Mathura-281406 (U.P.)

Declaration

I hereby declare that the work which is being presented in the M.C.A. Project **“Efficient Deepfake Face Detection Model Based on CNN and MobileNetV2 Transfer Learning: Intelligent Legal Documents Processing Using Retrieval Augmented Generation”**, in partial fulfillment of the requirements for the award of the *Master of Computer Applications* and submitted to the Department of Computer Engineering and Applications of GLA University, Mathura, is an authentic record of our own work carried under the supervision of **Dr. Sachendra Singh Chauhan, Assistant Professor.**

The contents of this project report, in full or in parts, have not been submitted to any other Institute or University for the award of any degree.

Sign _____
Name: Aditya Gaur
University Roll No.: 12584200012

Sign _____
Name: Vipin Kumar
University Roll No.: 12584200209

Sign _____
Name: Piyush Sharma
University Roll No.: 12584200138

Sign _____
Name: Jatin
University Roll No.: 12584200089

Certificate

This is to certify that the above statements made by the candidate are correct to the best of my/our knowledge and belief.

Supervisor
(Dr. Sachendra Singh Chauhan)
Designation of Supervisor
Dept. of Computer Engg. & App.

Project Co-ordinator
(Dr. Sachendra Singh Chauhan)
Assistant Professor
Dept. of Computer Engg. & App.
Date:

Program Co-ordinator
(Mr. Siddharth Singh)
Assistant Professor
Dept. of Computer Engg. & App.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our supervisor, Dr. Sachendra Singh Chauhan, for his continuous guidance, valuable suggestions, and patient encouragement throughout the development of this project. The successful completion of this project has been possible due to his constant support, insightful feedback, and motivation at every stage of the work.

We also wish to acknowledge the faculty members of the Department of Computer Engineering & Applications, GLA University, for their academic guidance and for providing an environment that encouraged systematic learning and disciplined work.

We extend our heartfelt thanks to our families and friends for their understanding, encouragement, and unwavering support during the preparation of this report and the development of the application.

Group Members:

Aditya Gaur

University Roll No.: 12584200012

Piyush Sharma

University Roll No.: 12584200138

Jatin

University Roll No.: 12584200089

Vipin Kumar

University Roll No.: 12584200209

ABSTRACT

Deepfake technology has advanced rapidly with the growth of artificial intelligence, making it increasingly difficult to distinguish between real and manipulated media. These synthetic creations pose significant risks, including the spread of misinformation, digital fraud, identity theft, and other forms of cybercrime. As a result, there is an urgent need for reliable and automated systems capable of detecting deepfake content accurately. This project introduces **DeepShield**, an efficient deepfake face detection system that leverages deep learning techniques. The system incorporates two models: a custom-built Convolutional Neural Network (CNN) and a pretrained MobileNetV2 model. Both models are trained using publicly available datasets, including FaceForensics and the Real vs Fake 10K dataset, ensuring diverse and realistic training data.

Among the two, the MobileNetV2 model achieved a higher accuracy of approximately 87%, outperforming the CNN in both efficiency and predictive performance. Its lightweight architecture makes it particularly suitable for real-time applications without compromising accuracy. DeepShield is deployed through a user-friendly web interface, with React handling the frontend and Flask managing the backend operations. This setup allows users to upload images easily and receive instant predictions indicating whether the content is authentic or manipulated.

–The system is designed to be scalable, efficient, and accessible. Furthermore, it has the potential for future enhancements, such as extending its capabilities to detect deepfakes in video content, thereby broadening its applicability in combating emerging digital threats.

Contents

Declaration	iii
Certificate	iv
Acknowledgement	v
Abstract	vi
Chapter 1 Motivation and Overview	
1.1 Introduction	1
1.2 Problem Statement	2
1.3 Objectives of the Project	3
1.4 Summary of Similar Applications	4
1.5 Scope and Significance	5
1.6 Organization of the Project Report	6
Chapter 2 Advantages of Transfer Learning:	
2.1 Introduction to Deepfake Detection	7
2.2 Traditional Techniques	8
2.3 CNN-Based Approaches	9
2.4 Transfer Learning and MobileNetV2	10
2.5 Comparison of Methods	11
2.6 Challenges in Deepfake Detection	12
2.7 Summary	13
Chapter 3 Overall System Architecture	18
Chapter 4 Implementation	20
Chapter 5 Software Testing	26
Chapter 6 Conclusion and Future Scope	32
References	35
Appendices	36

List of Figures

Figure 3.1: Deepfake Detection System Architecture	23
Figure 3.2: System Workflow Diagram	24
Figure 3.3: DFD Level 0 (Context Diagram)	25
Figure 3.4: DFD Level 1	26
Figure 3.5: CNN Model Architecture	27
Figure 3.6: MobileNetV2 Model	28
Figure 3.7: Use Case Diagram	29

List of Tables

Table 5.1: Functional Test Cases	32
Table 5.2: Integration Test Cases	33
Table 5.3: Performance Test Cases	34
Table A.1: Data Dictionary	35

Chapter 1

Introduction

1.1 Motivation and Overview

Deepfake technology refers to the use of deep learning algorithms to manipulate images in such a way that they appear realistic to human observers. These techniques are primarily based on neural networks that learn patterns from large datasets and generate synthetic content that closely resembles real data. With the rapid growth of artificial intelligence, deepfake generation has become more accessible and more advanced, making it increasingly difficult to distinguish between real and manipulated images. Although deepfake technology has useful applications in areas such as entertainment and media production, it also introduces serious risks. Manipulated images can be used to spread misinformation, create fake identities, and damage reputations. As these images become more realistic, human detection becomes unreliable, which increases the need for automated systems capable of identifying such manipulations. The increasing misuse of deepfake technology has created a demand for systems that can detect fake images accurately and efficiently. Traditional detection methods are no longer sufficient due to the high level of realism achieved by modern deepfake techniques. Therefore, this project focuses on developing a deep learning-based system that can automatically detect whether an image is real or fake.

The motivation behind this project arises from the growing concern over the misuse of deepfake technology. Fake images are being used in various harmful ways, including spreading false information, committing digital fraud, and manipulating public perception. These issues highlight the importance of developing tools that can verify the authenticity of digital content.

Another important motivation is the limitation of existing systems. Many available tools either require cloud-based processing or are designed for professional use, making them less accessible to general users. Additionally, uploading sensitive images to external servers raises privacy concerns. Therefore, there is a need for a system that is not only accurate but also user-friendly and privacy-focused.

This project aims to address these challenges by developing a system that performs deepfake detection efficiently while maintaining simplicity and accessibility. The goal is to provide a practical solution that can be used by individuals without requiring specialized technical knowledge

1.2 Problem Statement

The main problem addressed in this project is the lack of an accessible and reliable system for detecting deepfake images. While there has been significant research in this area, most solutions are either complex, expensive, or not suitable for general users.

Existing systems often rely on cloud-based platforms where users are required to upload images for processing. This raises concerns regarding data privacy and security. Additionally, many tools are designed for professional environments and are not easy to use for non-technical users.

From a technical perspective, detecting deepfake images is challenging because the differences between real and fake images are often very subtle. These differences may include slight distortions, lighting inconsistencies, or irregular textures, which are difficult to identify manually.

This project aims to solve these issues by developing a system that combines deep learning techniques with a user-friendly interface to provide accurate and real-time detection of deepfake images.

1.3 Objectives of The Project

The primary objective of this project is to develop a system that can accurately detect whether a facial image is real or fake using deep learning techniques. The system aims to provide reliable predictions while maintaining efficiency and ease of use.

Another important objective is to implement and compare two different approaches for deepfake detection. The first approach involves building a Convolutional Neural Network model from scratch, while the second approach uses a pretrained MobileNetV2 model through transfer learning. The comparison helps in identifying the most suitable model based on performance and efficiency.

The project also aims to integrate the trained model into a web-based application using Flask for the backend and React for the frontend. This allows users to interact with the system easily by uploading images and receiving instant results.

Additionally, the system is designed to ensure that processing is handled efficiently, providing quick responses without unnecessary complexity.

1.5 Scope And Significance

The scope of this project is focused on detecting deepfake images. The system is designed to classify images as either real or fake based on the features learned during training. It does not include video-based detection or identification of the specific technique used to create the deepfake.

The significance of this project lies in its practical application. It provides a simple and effective tool for detecting manipulated images, which can be useful in various scenarios such as digital content verification, educational purposes, and security applications.

The project demonstrates how deep learning can be applied to solve real-world problems related to digital media. It also highlights the importance of developing systems that are not only accurate but also accessible to a wider audience.

1.6 Organization of The Project Report

This report is organized into multiple chapters, each covering a different aspect of the project. The first chapter provides an introduction, motivation, problem statement, and objectives. The second chapter presents a review of existing techniques and research related to deepfake detection. The third chapter explains the system architecture and design of the proposed solution.

The fourth chapter describes the implementation details, including model development and integration with the web application. The fifth chapter focuses on testing and evaluation of the system. Finally, the sixth chapter presents the conclusion and discusses the future scope of the project.

Chapter 2

LITERATURE REVIEW

Deepfake detection has emerged as an important research area due to the rapid advancement of artificial intelligence and deep learning techniques. Deepfakes are typically generated using neural network-based methods that create highly realistic manipulated images. As these techniques have improved, it has become increasingly difficult to distinguish fake images from real ones using human observation alone. This has created a need for automated systems that can detect such manipulations effectively.

Various approaches have been proposed for deepfake detection. Earlier methods relied on traditional image processing techniques, which focused on identifying visual inconsistencies such as irregular textures, unnatural lighting, and distortions in facial regions. However, these approaches were limited in their effectiveness because modern deepfake techniques produce images with very few visible artifacts.

With the development of deep learning, Convolutional Neural Networks have become widely used for image classification tasks, including deepfake detection. CNN-based methods are capable of automatically learning important features from images. These models analyze patterns, textures, and structural details present in images to identify whether the image is real or manipulated. The ability of CNNs to extract hierarchical features makes them suitable for detecting subtle differences that are difficult to identify manually.

In CNN-based deepfake detection systems, the model processes the input image through multiple layers. Each layer extracts specific features, starting from basic elements such as edges and shapes and moving towards more complex patterns. This allows the model to detect inconsistencies such as blurred regions, irregular edges, and abnormal pixel distributions. Although CNN models provide good performance, they require large datasets and significant training time. Additionally, their performance may vary when tested on different datasets.

To improve efficiency and performance, transfer learning techniques are commonly used. Transfer learning involves using a pretrained model that has already been trained on a large dataset and adapting it to a specific task. In this project, MobileNetV2 is used as a pretrained model. It is designed to be lightweight and efficient, making it suitable for real-time applications.

MobileNetV2 uses a specialized architecture that reduces computational complexity while maintaining good accuracy. By using pretrained weights, the model already has knowledge of general image features, which helps in improving performance when applied to deepfake detection. The model is fine-tuned using the dataset to adapt it to the specific task of identifying manipulated images.

A comparison between CNN and MobileNetV2 shows that both approaches have their own advantages. CNN models provide strong feature extraction capabilities but require more training time and computational resources. On the other hand, MobileNetV2 offers better efficiency and faster processing while maintaining good accuracy. In this project, MobileNetV2 achieved an accuracy of approximately 87%, making it more suitable for the system.

Despite the progress in deepfake detection, several challenges remain. One of the major challenges is the high realism of deepfake images. Modern techniques produce images that are very close to real images, making detection difficult. Another challenge is the generalization problem, where models trained on one dataset may not perform well on another dataset.

Computational complexity is also an important issue, as some models require high processing power, which limits their use in real-time applications. Additionally, the availability of large and diverse datasets is limited, which affects the training process. Deepfake generation techniques are continuously improving, which makes it necessary to develop more robust detection systems.

Based on the analysis of existing techniques, this project adopts a combined approach by implementing both CNN and MobileNetV2 models. This allows for performance comparison and selection of the most effective model. The use of MobileNetV2 ensures better efficiency and performance, making it suitable for deployment in the system.

Chapter 3

OVERALL SYSTEM ARCHITECTURE

3.1 Introduction

This chapter describes the design of the proposed deepfake detection system, DeepShield. The system is developed using deep learning techniques to provide an efficient solution for detecting manipulated facial images. The architecture is designed to ensure smooth interaction between different components while maintaining performance and usability.

The system combines two approaches for detection, which include a custom Convolutional Neural Network and a pretrained MobileNetV2 model. These models are integrated into a web-based system that allows users to upload images and receive predictions in real time.

3.2 System Overview

DeepShield operates as a web-based application that enables users to upload facial images and receive predictions indicating whether the image is real or manipulated. The system is designed to handle the complete process of deepfake detection, starting from user input to final output display.

The system consists of three main components: the frontend, backend, and deep learning model. Each of these components plays a specific role in the functioning of the system. The frontend serves as the interface through which users interact with the application. It allows users to upload images, view previews, and receive results. The backend acts as the processing unit that manages data flow, performs preprocessing operations, and communicates with the trained model. The deep learning model is responsible for analyzing the input image and generating a prediction.

The system ensures that all components work together seamlessly. When a user uploads an image, it is immediately processed by the backend and passed to the model for analysis. The prediction is then sent back to the frontend and displayed to the user. This design of the system focuses on providing a smooth and efficient user experience. The interface is kept simple and intuitive so that users can easily interact with the system without any difficulty. At the same time, the backend and model are optimized to ensure accurate and fast predictions.

3.3 System Architecture

The architecture of DeepShield is designed to ensure efficient communication between different components while maintaining a clear and structured workflow. The system follows a layered approach, where each layer is responsible for a specific function.

The process begins at the frontend, where the user uploads an image using the web interface. The frontend is responsible for capturing user input and sending it to the backend in the form of a request. The uploaded image is transmitted to the backend server through an API.

Once the backend receives the image, it performs a series of preprocessing operations. These operations are necessary to prepare the image for analysis by the deep learning model. The preprocessing steps include resizing the image to a fixed dimension, normalizing pixel values, and ensuring that the image format is compatible with the model.

After preprocessing, the image is passed to the trained deep learning model. The model analyzes the image and extracts relevant features that help in identifying whether the image is real or fake. These features include patterns, textures, and inconsistencies present in the image.

Based on the extracted features, the model generates a prediction. The prediction indicates whether the image is authentic or manipulated, along with a confidence score. This confidence score represents the probability of the prediction being correct.

The result is then sent back to the backend, which forwards it to the frontend. The frontend displays the result to the user in a clear and understandable format. The entire workflow is designed to be efficient and fast, ensuring that users receive results without delay.

The architecture can be represented as a sequence of steps where data flows from the user to the frontend, then to the backend, followed by the model, and finally back to the user. This structured approach ensures that the system remains organized and easy to manage.

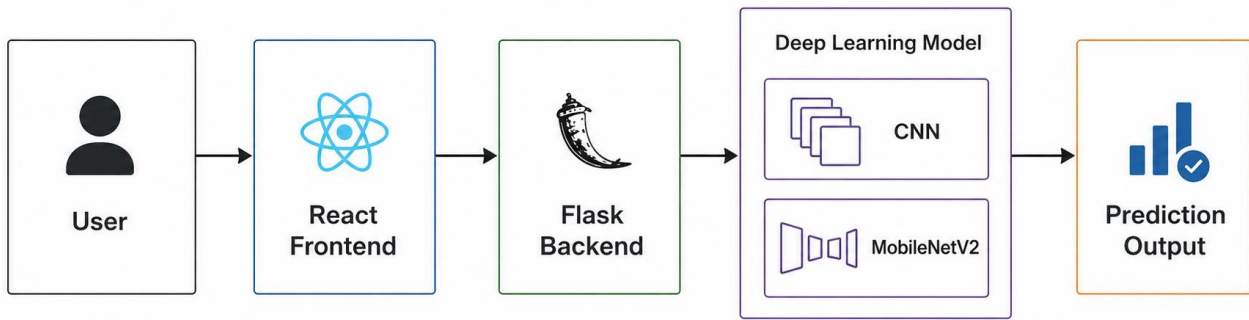


Figure 3.1: Deepfake Detection System Architecture

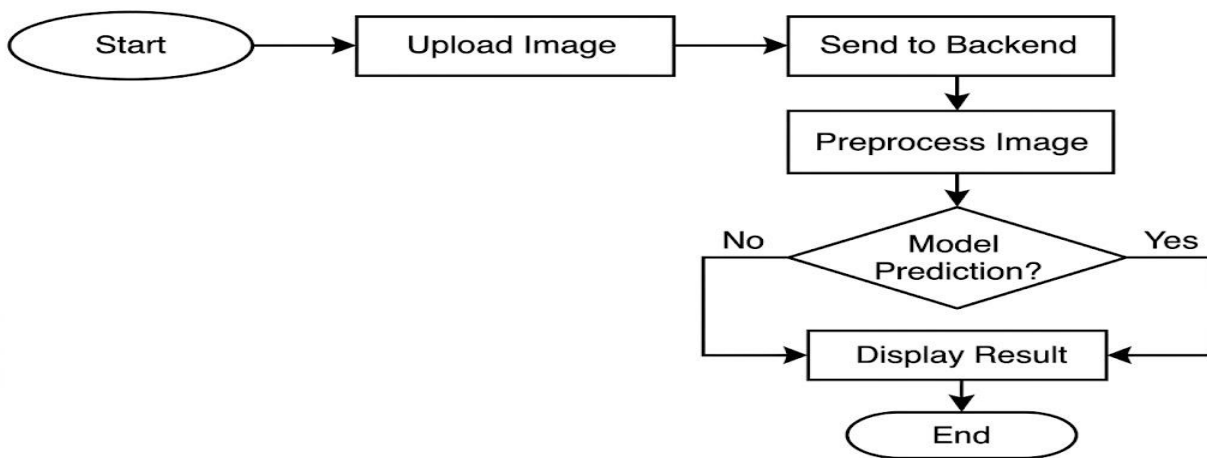


Figure 3.2: System Workflow Diagram

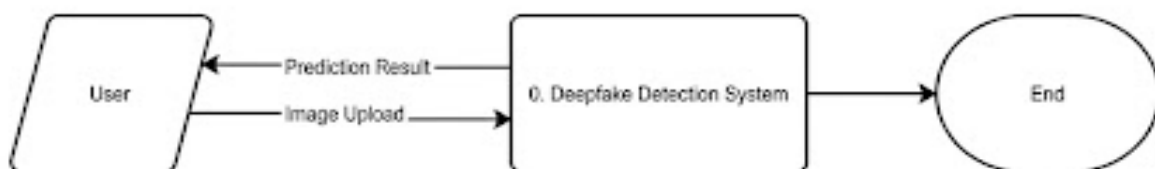


Figure 3.3: DFD Level 0 (Context Diagram)

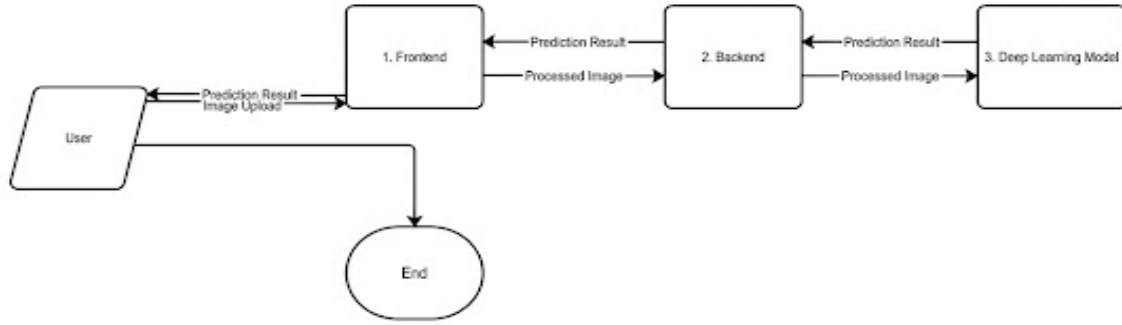


Figure 3.4: DFD Level 1

3.4 Dataset Description

The effectiveness of the deep learning models used in this project depends heavily on the quality and diversity of the dataset. In this system, publicly available datasets are used to train and evaluate the models.

The FaceForensics dataset is one of the primary datasets used in this project. It contains a large number of real and manipulated facial images. These images are generated using various deepfake techniques, making the dataset suitable for training models that can detect different types of manipulations. The dataset provides a wide range of variations in terms of facial expressions, lighting conditions, and image quality.

Another dataset used in this project is the Real vs Fake 10K dataset. This dataset contains a balanced collection of real and fake images, which helps in improving the performance of the model. The balanced nature of the dataset ensures that the model does not become biased toward a particular class.

3.5 Data Preprocessing

Data preprocessing is an essential step in the deep learning pipeline. It ensures that the input data is consistent and suitable for model training and prediction.

In this system, all images are resized to a fixed dimension. This is necessary because the deep learning model requires input images of a specific size. Resizing ensures uniformity and allows the model to process images efficiently.

Normalization is another important preprocessing step. Pixel values are scaled to a standard range, which helps in stabilizing the training process and improving model performance. Without normalization, the model may face difficulty in learning patterns effectively.

The dataset is divided into training and testing sets. The training set is used to train the model, while the testing set is used to evaluate its performance. This separation ensures that the model is tested on unseen data, providing a reliable measure of its accuracy.

3.6 Proposed Models

The system uses two different models for deepfake detection. The first model is a Convolutional Neural Network that is designed specifically for this task. The CNN model consists of multiple layers, including convolution layers, activation functions, pooling layers, and fully connected layers. These layers work together to extract features from the input image and classify it as real or fake.

The second model is MobileNetV2, which is a pretrained model used through transfer learning. MobileNetV2 is designed to be efficient and lightweight, making it suitable for real-time applications. The model is fine-tuned using the dataset to adapt it to the deepfake detection task. Both models are trained and evaluated, and their performance is compared. The comparison helps in selecting the most suitable model for the system.

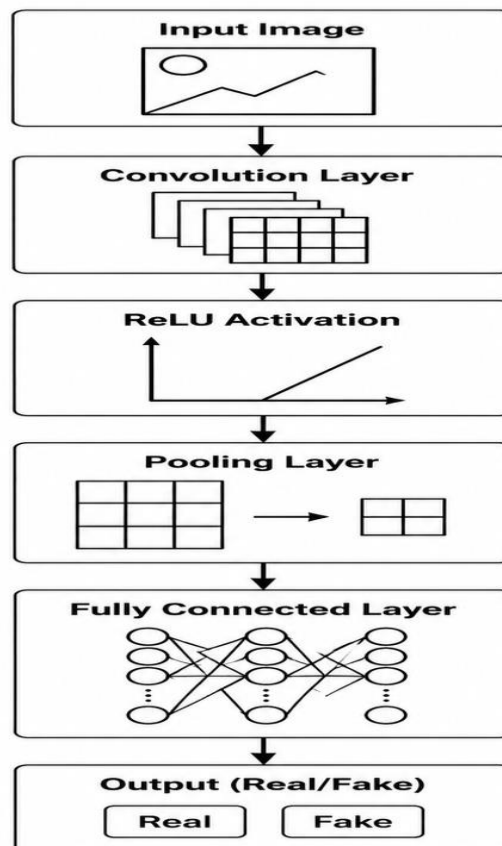


Figure 3.5: CNN Model Architecture

Transfer Learning with MobileNetV2

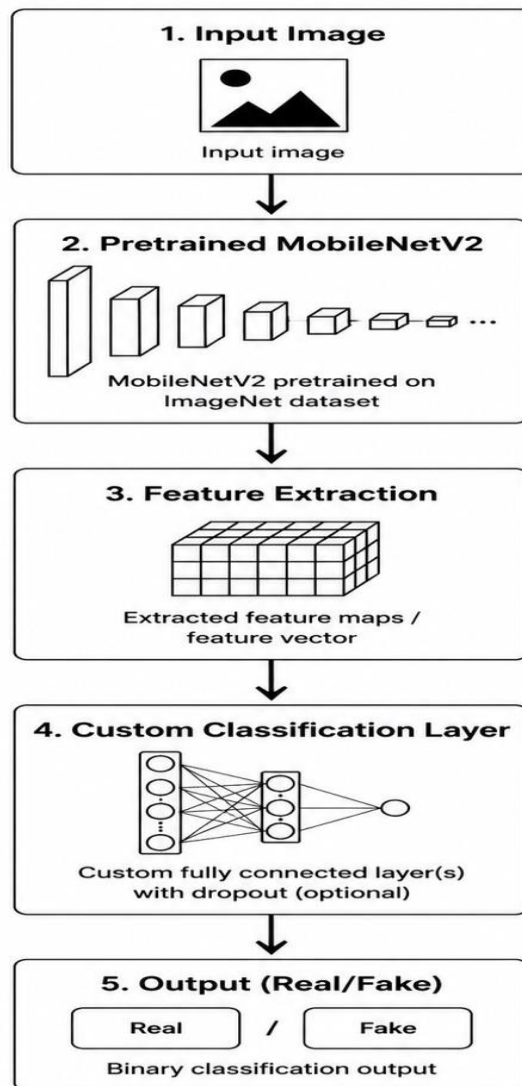


Figure 3.6: MobileNetV2 Model

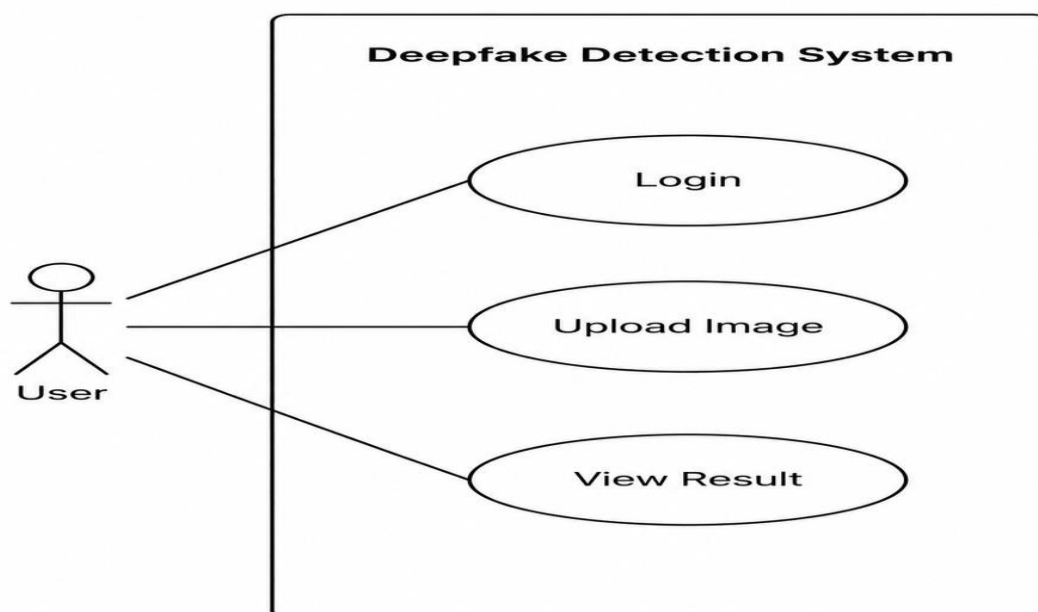


Figure 3.7: Use Case Diagram

3.7 Model Training Process

The model training process involves several steps. First, the dataset is loaded and preprocessed. The model is then initialized, and training is performed using the training data.

During training, the model learns to identify patterns and features that distinguish real images from fake ones. The performance of the model is evaluated using metrics such as accuracy, precision, recall, and loss.

The training process continues until the model achieves satisfactory performance.

3.8 Model Comparison

The CNN model and MobileNetV2 model are compared based on various parameters such as accuracy, speed, and efficiency. The results show that MobileNetV2 performs better in terms of accuracy and computational efficiency.

MobileNetV2 requires less training time and provides faster predictions, making it more suitable for real-time applications. Therefore, it is selected as the final model for the system.

3.9 System Implementation Strategy

The system is implemented in multiple stages, starting from data collection and model development to frontend and backend integration. Each stage is carefully executed to ensure proper functioning of the system.

3.10 Use Case Description

The primary use case of the system is image verification. The user uploads an image, and the system processes it to determine whether it is real or fake.

3.11 Advantages of Proposed System

The system provides real-time detection, high accuracy, and a simple user interface. It is efficient and easy to use.

3.12 Limitations

The system is limited to image-based detection and does not support video detection. Its performance depends on the quality of the dataset.

Chapter 4

IMPLEMENTATION

4.1 Introduction

This chapter describes the implementation of the proposed deepfake detection system, DeepShield. It explains how the system is developed by integrating deep learning models with web technologies to create a functional and user-friendly application. The implementation phase focuses on transforming the system design into a working model that can process user inputs and generate predictions in real time.

The system is implemented using a combination of deep learning frameworks and web development technologies. The goal of the implementation is to ensure that the system performs efficiently while maintaining accuracy and usability. Each component of the system, including the model, backend, and frontend, is developed and integrated carefully to achieve seamless operation.

4.2 Development Environment

The development of DeepShield requires both hardware and software support to ensure smooth execution of the system. The hardware configuration is selected to handle model training and application execution efficiently, while the software environment provides the necessary tools and frameworks for development.

The system is developed on a machine with a processor capable of handling computational tasks required for deep learning. Adequate memory is required to process image data and train the models effectively. Storage is also an important factor, as datasets and model files require sufficient space. Although a GPU is optional, it can significantly improve the speed of model training.

The software environment plays a crucial role in the development process. The system is developed using Python as the primary programming language. Libraries such as TensorFlow and Keras are used for building and training deep learning models. NumPy is used for numerical operations, while Matplotlib is used for visualization purposes during training.

The backend of the system is implemented using Flask, which provides a lightweight framework for handling requests and integrating the model. The frontend is developed using React, which allows the creation of a responsive and interactive user interface. Development tools such as code editors are used to write and manage the code efficiently.

4.3 Model Implementation

The implementation of deep learning models is a critical part of the system. Two models are developed in this project: a custom Convolutional Neural Network and a pretrained MobileNetV2 model. Both models are trained and evaluated to determine their performance in detecting deepfake images.

4.3.1 CNN Model Implementation

The Convolutional Neural Network is designed specifically for the task of image classification. It is built using multiple layers that work together to extract features from input images and classify them as real or fake.

The implementation of the CNN model involves the following steps:

- Defining convolutional layers to extract features from images
- Applying activation functions to introduce non-linearity
- Using pooling layers to reduce dimensionality
- Flattening the output to prepare it for classification
- Adding fully connected layers for final prediction

The CNN model processes input images by passing them through these layers. Each layer extracts specific features, starting from basic patterns and progressing to more complex structures. The model is trained using labeled data, allowing it to learn the differences between real and fake images.

Although the CNN model provides good feature extraction capabilities, it requires more training time and computational resources compared to pretrained models.

4.3.2 MobileNetV2 Implementation

MobileNetV2 is implemented using transfer learning. It is a pretrained model that has already been trained on a large dataset, allowing it to learn general image features.

The implementation process includes:

- Loading the pretrained MobileNetV2 model
- Removing the final classification layer
- Adding a custom classification layer for deepfake detection
- Training the model on the dataset

MobileNetV2 uses an efficient architecture that reduces computational complexity while maintaining good performance. By using pretrained weights, the model requires less training time and provides better accuracy.

During training, the model is fine-tuned to adapt to the specific task of deepfake detection. This approach improves the overall performance of the system.

4.4 Backend Implementation (Flask)

The backend of the system is implemented using Flask, which handles communication between the frontend and the deep learning model. It acts as the core processing unit of the system.

The backend performs several important functions:

- Receiving image input from the frontend
- Performing preprocessing on the image
- Loading the trained model
- Generating predictions
- Sending results back to the frontend

The workflow of the backend can be described as follows:

When a user uploads an image, the request is sent to the Flask server. The server receives the image and applies preprocessing steps such as resizing and normalization. The processed image is then passed to the model for prediction. Once the model generates the result, the backend sends the response back to the frontend.

The use of Flask ensures that the system remains lightweight and efficient. It also simplifies the integration of the model with the web application.

4.5 Frontend Implementation (React)

The frontend of the system is developed using React, which provides a responsive and interactive user interface. The design focuses on simplicity and ease of use, allowing users to interact with the system without difficulty.

The frontend includes the following features:

- A clean and simple user interface
- Image upload functionality
- Drag-and-drop support
- Image preview before submission
- Real-time display of results

The React application communicates with the Flask backend using API requests. When a user uploads an image, the frontend sends the image to the backend and waits for the response. Once the result is received, it is displayed to the user.

The use of React ensures smooth interaction and fast response times, enhancing the overall user experience.

4.6 Login Page

The system includes a login page that provides secure access to users. This feature ensures that only authorized users can use the application.

The login page includes:

- Input fields for email and password
- A sign-in option
- A simple and user-friendly interface

This feature helps in maintaining security and prevents unauthorized access to the system.

4.6.5 Image Detection Interface

The image detection interface is the main component of the application where users interact with the system.

Key features include:

- Image upload option
- Drag-and-drop functionality
- Image preview display
- Detection result output

Users can upload an image, and the system processes it to generate a prediction. The interface is designed to make this process simple and intuitive.

4.6.6 Result Display

The result display shows the output generated by the model. It provides clear information about the prediction.

The system displays:

- Whether the image is real or fake
- Confidence score of the prediction
- Preview of the uploaded image

For example, the system may display a result such as “Authentic” with a confidence score. This helps users understand the reliability of the prediction.

4.7 Advantages of Implementation

The implementation of the system provides several advantages. It allows real-time detection of deepfake images and provides accurate results. The system is easy to use and does not require technical expertise. The integration of deep learning models with a web interface makes the system practical and accessible.

4.8 Limitations

Despite its advantages, the system has certain limitations. It is limited to image-based detection and does not support video analysis. The accuracy of the system depends on the quality and diversity of the dataset used for training.

4.9 Summary

This chapter explained the implementation of the DeepShield system in detail. It covered model development, backend and frontend integration, and user interface design. The system successfully combines deep learning with web technologies to provide an efficient solution for deepfake detection.

4.10 Project Execution Evidence

The following figure shows the geo-tagged image captured during the project development phase, indicating the team's presence and active participation.



Figure 4.4: Geo-tagged Image of Project Team

Chapter 5

Software Testing

5.1 Introduction

Software testing is an essential phase in the development of any system, as it ensures that the application functions correctly and meets the specified requirements. In the case of the DeepShield system, testing plays a crucial role in verifying the accuracy of deepfake detection, the reliability of the system components, and the overall performance of the application.

The DeepShield system integrates multiple components, including the frontend interface, backend processing, and deep learning models. Therefore, it is necessary to test each component individually as well as the system as a whole. The testing process ensures that the system is capable of handling user inputs, processing images correctly, and generating accurate predictions in real time.

Testing also helps in identifying errors, improving system stability, and ensuring that the application provides a smooth user experience. By performing thorough testing, the system can be validated before deployment.

5.2 Testing Objectives

The primary objective of testing in this project is to ensure that the system performs its intended function of detecting deepfake images accurately and efficiently. The testing process is designed to verify both functional and non-functional aspects of the system.

The key objectives of testing include ensuring that the system correctly classifies images as real or fake, validating that the frontend and backend are properly integrated, and confirming that the application responds correctly to user inputs. Additionally, testing aims to evaluate the performance of the deep learning model and ensure that it produces reliable predictions.

Another important objective is to identify and resolve any errors or issues present in the system. This includes checking for incorrect outputs, system crashes, or delays in processing. Testing also ensures that the user interface functions properly and provides a seamless experience.

5.3 Testing Methodology

The testing of the DeepShield system is carried out using a systematic approach to ensure complete validation of the application. The process includes testing individual components as well as the entire system.

The testing methodology involves the following approaches:

- **Unit Testing:** Each component of the system, such as the backend functions and model prediction module, is tested individually to ensure correct functionality.
- **Integration Testing:** The interaction between frontend, backend, and model is tested to ensure smooth data flow.
- **System Testing:** The complete system is tested as a whole to verify overall performance and functionality.
- **Black Box Testing:** The system is tested without considering internal code structure, focusing only on input and output behavior.

This structured approach ensures that all aspects of the system are thoroughly tested.

5.4 System Testing

System testing is performed to ensure that all components of the DeepShield system work together correctly. This type of testing verifies the complete workflow of the application, from image upload to result display.

During system testing, the following features are tested:

- Image upload functionality
- Backend processing of images
- Model prediction accuracy
- Response time of the system

Display of results on the frontend

The system is tested using different types of images, including real and fake images, to evaluate its performance under various conditions. The results show that the system is able to process images efficiently and provide predictions in real time.

5.5 Test Cases

Test cases are designed to verify the correctness of the system under different scenarios. Each test case includes an input, expected output, and actual result.

Table 5.1: Functional Test Cases

Test Case ID	Input Type	Description	Expected Output	Actual Output	Status
TC01	Real Image	Upload a genuine image	Real	Real	Pass
TC02	Fake Image	Upload a manipulated image	Fake	Fake	Pass
TC03	Invalid File	Upload non-image file	Error Message	Error Message	Pass
TC04	Large Image	Upload high-resolution image	Process Image	Processed	Pass
TC05	No Input	Submit without uploading image	Warning	Warning	Pass

Table 5.2: Integration Test Cases

Test Case ID	Description	Expected Result	Status
IT01	Frontend to backend communication	Image successfully transmitted	Pass
IT02	Backend to model interaction	Model processes image correctly	Pass
IT03	Model to frontend response	Result displayed correctly	Pass

Test Case ID	Scenario	Expected Outcome	Status
PT01	Multiple image uploads	Stable performance	Pass
PT02	Real-time prediction	Quick response	Pass
PT03	Continuous usage	No system crash	Pass

Table 5.3: Performance Test Cases

5.6 Security and Robustness Testing

Security testing ensures that the system is protected against unauthorized access and improper usage. The login feature is tested to ensure that only valid users can access the system.

Robustness testing checks how the system behaves under unexpected conditions. This includes handling invalid inputs, corrupted files, and incomplete data. The system is designed to handle such cases gracefully by displaying appropriate error messages instead of crashing.

5.7 Testing Results and Analysis

The results of testing indicate that the DeepShield system performs effectively in detecting deepfake images. The model provides accurate predictions for both real and fake images. The integration between frontend and backend works smoothly, ensuring efficient data flow.

The response time of the system is satisfactory, allowing users to receive results quickly. The user interface functions properly, and all features operate as expected. Minor issues encountered during testing were resolved, improving the overall stability of the system.

5.8 Summary

This chapter discussed the testing process of the DeepShield system. Various testing methods were used to ensure that the system functions correctly and efficiently. The results show that the system is reliable and capable of performing deepfake detection in real time. The successful completion of testing confirms that the system is ready for deployment.

Chapter 6

Conclusion and Future Scope

6.1 Conclusion

This chapter presents the conclusion of the DeepShield project and summarizes the overall work carried out during the development of the deepfake detection system. It highlights the key outcomes of the project, the effectiveness of the implemented models, and the overall performance of the system. In addition, this chapter discusses the future scope of the project, identifying possible improvements and extensions that can enhance the capabilities of the system.

The DeepShield system was successfully developed to detect deepfake images using deep learning techniques. The project focused on building an efficient and user-friendly application capable of identifying manipulated facial images with reliable accuracy. The system integrates deep learning models with a web-based interface, enabling users to upload images and receive predictions in real time.

Two different models were implemented as part of this project: a custom Convolutional Neural Network and a pretrained MobileNetV2 model using transfer learning. Both models were trained and evaluated using publicly available datasets containing real and fake images. The comparison between these models showed that MobileNetV2 performed better in terms of accuracy and efficiency. It achieved an accuracy of approximately 87%, making it more suitable for real-time applications.

The implementation of the system demonstrates how deep learning techniques can be effectively applied to solve real-world problems related to digital media authentication. The integration of the model with a frontend and backend system ensures smooth operation and provides a seamless user experience. Users can easily interact with the system without requiring technical knowledge, which makes the application accessible to a wider audience.

The system performs all operations efficiently, from image upload to prediction display. The preprocessing steps ensure that input images are properly formatted, while the model accurately classifies them based on learned features. The results obtained during testing indicate that the system is reliable and capable of handling real-time detection tasks.

Overall, the project achieves its objective of developing a practical and efficient deepfake detection system. It highlights the importance of artificial intelligence

in addressing modern challenges related to digital security and content verification.

6.2 Achievements of the Project

The successful completion of the DeepShield system demonstrates several key achievements. The project effectively implements deep learning models for image classification and integrates them into a web-based application. The use of both CNN and MobileNetV2 models allows for performance comparison and selection of the most efficient approach.

The system provides real-time detection of deepfake images, which is a significant achievement considering the complexity of the task. The user-friendly interface ensures that the system can be used easily without requiring technical expertise. The successful integration of frontend and backend components further enhances the functionality of the system.

Another important achievement is the use of publicly available datasets for training the models, which ensures that the system is developed using realistic and diverse data. The testing phase confirms that the system performs accurately and efficiently under different conditions.

6.4 Limitations

Despite its successful implementation, the DeepShield system has certain limitations. The system is currently limited to image-based deepfake detection and does not support video analysis. Since deepfake technology is widely used in videos, extending the system to handle video data would improve its applicability.

The accuracy of the system depends on the quality and diversity of the dataset used for training. If the dataset does not cover certain types of manipulations, the model may not perform well in those cases. Additionally, the system provides predictions based on learned patterns and may not always guarantee perfect accuracy.

Another limitation is that the system does not identify the specific technique used to create the deepfake. It only classifies the image as real or fake, without providing detailed analysis of the manipulation.

6.5 Future Scope

The DeepShield system provides a strong foundation for further development and improvement. Several enhancements can be made to extend its capabilities and improve its performance.

One of the major areas for future work is the extension of the system to support video-based deepfake detection. This would allow the system to analyze video frames and detect manipulated content in real time. Implementing such functionality would significantly increase the usefulness of the system.

Another possible improvement is the use of larger and more diverse datasets. Training the model on a wider range of data can improve its generalization ability and increase accuracy. Continuous updates to the dataset can help the system adapt to new deepfake techniques.

The system can also be improved by enhancing the user interface and adding additional features such as history tracking of uploaded images. This would allow users to review previous results and manage their data more effectively.

Further improvements can include optimizing the model for faster performance and reducing processing time. This would make the system more efficient and suitable for real-time applications.

6.6 Final Remarks

Deepfake detection is an important and challenging problem in today's digital world. The increasing use of manipulated media highlights the need for reliable detection systems that can verify the authenticity of digital content.

The DeepShield system provides an effective solution by combining deep learning techniques with a user-friendly web interface. It demonstrates how advanced technologies can be used to address real-world challenges and improve digital security.

The project not only achieves its objectives but also provides a foundation for future research and development in the field of deepfake detection. With further improvements and enhancements, the system can be extended to handle more complex scenarios and provide even better performance.

REFERENCES

1. Ian Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.
2. Alec Radford, L. Metz, and S. Chintala, “Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks,” 2016.
3. Andrew G. Howard et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” 2017.
4. Mark Sandler et al., “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” 2018.
5. TensorFlow, “TensorFlow Documentation.” Available: <https://www.tensorflow.org>
6. Keras, “Keras Documentation.” Available: <https://keras.io>
7. Flask, “Flask Documentation.” Available: <https://flask.palletsprojects.com>
8. React, “React Documentation.” Available: <https://reactjs.org>
9. NumPy, “NumPy Documentation.” Available: <https://numpy.org>
10. Matplotlib, “Matplotlib Documentation.” Available: <https://matplotlib.org>
11. FaceForensics Dataset, “FaceForensics++: Learning to Detect Manipulated Facial Images,” 2019.
12. Real vs Fake 10K Dataset, Public Dataset for Real and Fake Image Classification.

APPENDICES

Data Dictionary

The data dictionary provides a structured description of the data elements used in the DeepShield system. It helps in understanding how data is handled, processed, and transferred between different components of the system. This section defines the type, format, and purpose of each data element involved in the deepfake detection process.

In this system, the primary input is the image uploaded by the user. This image is processed by the backend and converted into a suitable format before being passed to the deep learning model. The model then generates a prediction along with a confidence score, which is displayed to the user.

Table A.1: Data Dictionary

Field Name	Data Type	Description
Image File	File	Image uploaded by the user
Image Path	String	Location where image is stored
Processed Image	Array	Image after preprocessing
Prediction	String	Output result (Real or Fake)
Confidence Score	Float	Probability of prediction

Sample Input and Output

This section presents sample inputs and outputs of the DeepShield system. It helps in demonstrating how the system processes images and generates predictions. The examples provided here give a clear understanding of the working of the system.

The input to the system is a facial image uploaded by the user. After processing, the system classifies the image as either real or fake. Along with the classification result, a confidence score is displayed.

DS

DeepShield

Features

About

Demo

Sign in

Get Started

Powered by Advanced AI

Detect Deepfakes with Confidence

DeepShield uses cutting-edge AI technology to detect deepfake images instantly. Protect yourself from misinformation and verify image authenticity in seconds.

Try Now

Watch Demo

90 - 99.8% Detection Accuracy

Lightning Fast Analysis

Enterprise Security

Real-time Detection

Analyze images instantly with our advanced neural networks that detect even the most sophisticated deepfakes.

Advanced AI

Powered by state-of-the-art machine learning models trained on millions of images for superior accuracy.

100% Private

Your images are never stored or shared. All analysis happens securely on our protected servers.

DS

DeepShield

Protecting the world from deepfakes with AI.

Product

Features

Pricing

Demo

Company

About

Blog

Contact

Legal

Privacy

Terms

Security

© 2024 DeepShield. All rights reserved.

DS

DeepShield

Features

About

Demo

Sign in

Get Started

Watch Our Demo

See DeepShield in action - our advanced deepfake detection technology protecting digital authenticity.

Project Overview

Understanding DeepShield's mission and capabilities

What is DeepShield?

DeepShield is an advanced deepfake detection platform that uses cutting-edge AI and machine learning algorithms to identify manipulated media content. Our technology analyzes videos, images, and audio to detect signs of artificial generation or manipulation.

• Real-time detection capabilities

• High accuracy across multiple media types

• User-friendly web interface

• Continuous learning and improvement

Key Features

Multi-Modal Analysis

Combines visual, audio, and metadata analysis for comprehensive detection

Scalable Architecture

Built to handle large volumes of content efficiently

Privacy-First Design

Processes content locally without compromising user privacy

Live Demo

Watch DeepShield detect deepfakes in real-time

Play

Demo video will be embedded here

Replace with actual video URL or embed code

This demo showcases our detection algorithms processing various types of media content.

Ready to Try DeepShield?

Experience the power of AI-driven deepfake detection firsthand.

Get Started

Learn More

DS

DeepShield

Protecting the world from deepfakes with AI.

Product

Features

Pricing

Demo

Company

About

Blog

Contact

Legal

Privacy

Terms

Security

© 2024 DeepShield. All rights reserved.

Dept. of CEA, GLAU, Mathura

28

Welcome Back

Sign in to your account to continue

Email
gauradityas111@gmail.com

Password
.....

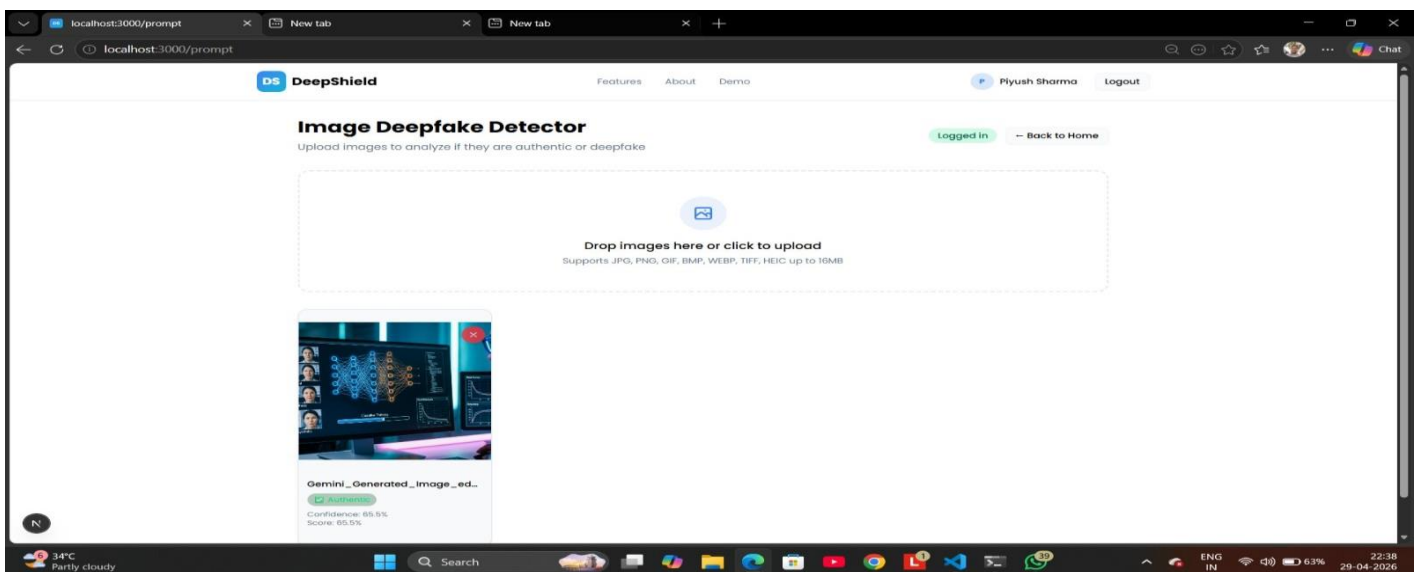
[Sign In](#)

Don't have an account? [Sign up](#)

Demo Credentials:
Email: demo@example.com
Password: demo123

N

This section includes additional screenshots of the system interface that are not covered in the main chapters. These screenshots provide a better understanding of how the system looks and functions during execution.



This section provides details about the training process of the models used in the DeepShield system. The models are trained using datasets containing both real and fake images.

The training process includes preprocessing of images, model initialization, and training using labeled data. The performance of the models is evaluated using accuracy and loss metrics. The

MobileNetV2 model achieved better performance compared to the CNN model and is therefore used as the final model in the system.